Final Exam — Sample C

Name: _____

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	150 minutes.
Length	Seven questions, one to two per category.
Coverage	Lectures 5.1 through 10.1 (JS/TS and the browser, React, deployment, Docker, CI/CD, concurrency, security, agents), plus assignments 3 and 4.
Categories	Trace-through, spot the bug, specification, code review, short design.
Grading	Curved. The exam is intentionally hard; expect to leave some parts unfinished.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks you to write working code from scratch on paper would punish you for taking the course seriously. Instead, every question is built around a skill the agent can't do for you: reading code carefully, finding bugs by inspection, writing the spec you'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

HOW TO USE THIS SAMPLE

This is a representative exam, not the actual one — the real final will have seven questions in the same five categories at roughly the same difficulty. Solutions are in a separate document; try the questions cold first, then check.

Question 1

Trace-through

This is the agent loop from lecture. Assume `read_file("notes.txt")` returns the string `"buy milk"`. The model replies twice: its first reply is a single `tool_use` block (id `tu_1`) calling `read_file` with `{"path": "notes.txt"}` and `stop_reason == "tool_use"`; its second reply is one text block `"Your notes say: buy milk"` with `stop_reason == "end_turn"`.

```
messages = [{"role": "user", "content": "What's in notes.txt?"}]

while True:
    r = client.messages.create(model="claude-opus-4-8", max_tokens=1024,
                               tools=tools, messages=messages)
    messages.append({"role": "assistant", "content": r.content})
    if r.stop_reason != "tool_use":
        print(r.content[0].text)
        break
    for block in r.content:
        if block.type == "tool_use":
            result = read_file(block.input["path"])
            messages.append({
                "role": "user",
                "content": [{"type": "tool_result",
                             "tool_use_id": block.id,
                             "content": result}],
            })
```

1. How many times is `client.messages.create` called, and why does the loop stop when it does?
2. Write out the `messages` list at the instant `print` runs: how many entries, and the role + gist of each.
3. The model has no memory between calls. What makes the second call able to use the file's contents? Point at the exact mechanism in this code.

Question 2

Trace-through

An event has exactly **one** seat left. Two buyers hit `buy_ticket` at nearly the same instant; both run their SELECT before either runs its UPDATE.

```
def buy_ticket(event_id, user_id):
    row = db.execute(
        text("SELECT seats_left FROM events WHERE id = :e"),
        {"e": event_id},
    ).first()
    if row.seats_left <= 0:
        raise HTTPException(409, "sold out")
    db.execute(text("UPDATE events SET seats_left = seats_left - 1 WHERE id = :e"),
               {"e": event_id})
    db.execute(text("INSERT INTO tickets (event_id, user_id) VALUES (:e, :u)"),
               {"e": event_id, "u": user_id})
```

1. Trace what happens. What is `seats_left` afterward, and how many tickets exist?
2. A teammate wraps each request in `with db.engine.begin() as conn:`. Does that fix it? Explain.
3. Give a fix that is atomic, and explain why it holds regardless of timing.

Question 3

Spot the bug

Your agent produced this list. It works in a demo. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```
function TodoList({ initial }: { initial: Todo[] }) {
  const [todos, setTodos] = useState(initial);

  function toggle(i: number) {
    todos[i].done = !todos[i].done;
    setTodos(todos);
  }

  return (
    <ul class="todos">
      {todos.map((t, i) => (
        <li key={i} onClick={() => toggle(i)}>
          {t.done ? "✓" : "○"} {t.title}
        </li>
      ))}
    </ul>
  );
}
```

Question 4

Spot the bug

Your agent produced these two endpoints. Read them carefully.

```
@app.get("/api/posts/search")
def search_posts(q: str):
    rows = db.execute(
        text(f"SELECT id, message FROM posts WHERE message LIKE '%{q}%'")
    ).mappings().all()
    return [dict(r) for r in rows]

@app.post("/api/posts/{post_id}/export")
def export_post(post_id: int, fmt: str):
    os.system(f"pandoc post_{post_id}.md -o out.{fmt}")
    return {"ok": True}
```

1. Name the vulnerability in `search_posts`, give a concrete `q` that exploits it, and the fix.
2. Name the vulnerability in `export_post`, give a concrete `fmt` that exploits it, and the fix.
3. Both bugs share one root cause. State it in one sentence.

Question 5

Specification

Feature request to your agent: “Autosave the draft as the user types.” The first cut is below and it works in a quick demo. Before you let it ship, write the spec you’d hand back. Do not write code.

```
editor.addEventListener("input", async () => {
  await fetch(`/api/drafts/${draftId}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text: editor.value }),
  });
  status.textContent = "Saved";
});
```

1. The save states this UI must show the user — name them.
2. Two behaviors it gets wrong for a fast typist on a flaky network.
3. One product decision that is ambiguous, and the call you would make.

Question 6

Code review

A teammate's PR adds this Dockerfile for the full-stack BBS (FastAPI serving the built React app), deployed on Railway. Their PR note: "Builds fine. Image is 1.4 GB and every code change takes ~3 min to rebuild. LGTM?"

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
RUN cd frontend && npm install && npm run build
EXPOSE 8000
CMD uvicorn app:app --host 0.0.0.0 --port 8000
```

1. Why are rebuilds slow after a one-line code change? Restructure so they aren't.
2. Why is the image 1.4 GB, and how do you shrink it substantially?
3. The `CMD` has a deploy bug for Railway, and one more production-readiness gap is missing. Name both.

Question 7

Short design

Your team is adding an agent to the BBS that answers questions and can take moderation actions. It's exposed in a chat box to end users, and it can read post contents — some written by untrusted users. Your agent wrote this tool setup.

```
tools = [
  {"name": "run_sql",  "description": "Run any SQL against the app database",
   "input_schema": {"type": "object", "properties": {"query": {"type": "string"}}}},
  {"name": "run_bash", "description": "Run a shell command",
   "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}}},
]

def run_bash(cmd):
    return subprocess.run(cmd, shell=True, capture_output=True, text=True).stdout
```

1. "A tool is authorized access." Explain what these two tools actually grant to whoever can talk to the agent, and why `run_bash` with `shell=True` is the worst of it.
2. Redesign the tools following "narrow beats broad," and say what else you'd add for the destructive ones.
3. Untrusted post text flows into the agent's context. Describe the attack and state the principle that actually contains it.